DNP3 Traffic Obfuscation on a Programmable Switch — an End-to-End Account

The problem, the platform, everything built and measured, the challenges, and what remains open

DNP3 / Case-A timing and size obfuscation — branch research/caseA-ditto-queue

2026-07-23

How to read this. Sections 1–3 are background — what the problem is and why, explained from scratch with examples. Sections 4–7 are what we actually did and measured, with the figures. Section 8 is the honest list of things that went wrong and what they taught us. Sections 9–10 place the work in the research literature and state the limits. Worked examples are followed byte-by-byte; honest caveats are flagged; `monospace paths` point to real files in the repository, and superscript-style numbers `[n]` point to the reference list.

1. The problem: fingerprinting a relay by its timing

DNP3 (Distributed Network Protocol 3, standardized as IEEE 1815 [1]) is one of the main languages that electric utilities use to talk to the equipment in substations. A control center runs a *master*; out in the field sits an *outstation* — here, a **SEL-751 feeder protection relay**, a real piece of hardware that protects a power line and reports measurements. The master *polls*; the outstation *responds*. That is the whole conversation: "tell me your status," "here it is."

> **Example — one DNP3 poll in plain terms.** The master sends a **Class-0 read**: "send me all your current static data." The relay answers with a response carrying, in our case, **69 data points** (breaker states, analog measurements). Over TCP this is: master sends a `READ` request, relay sends a bare TCP acknowledgement, then relay sends the DNP3 `RESPONSE`. Nothing is encrypted — DNP3 as deployed here has no confidentiality — so anyone who can see the wire sees the bytes and, crucially, *the timing*.

The threat: passive device fingerprinting

An attacker who can passively observe substation traffic wants to know *what device* is at each address, without ever sending a packet — reconnaissance for a later attack. Formby, Srinivasan, Leonard, Rogers and Beyah showed at NDSS 2016 [2] that industrial devices can be fingerprinted by a physical timing signature they cannot easily hide: the **Cross-Layer Response Time (CLRT)** — the gap between the low-level acknowledgement of a request and the actual application response. Different relays, running different firmware on different hardware, take characteristically different amounts of time to think. That timing is a fingerprint.

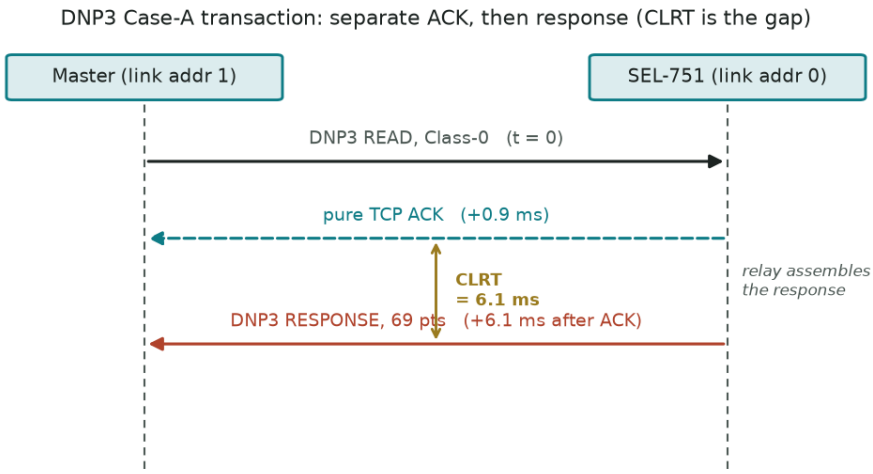

Figure 1: The DNP3 Case-A transaction. The relay first sends a bare TCP ACK, then — after some processing — the DNP3 response. The gap between them is the CLRT, the physical fingerprint.

> **The key definitions, once.** *Separate ACK (our "Case A")*: the relay first sends a bare TCP acknowledgement (no DNP3 payload), and only later sends the DNP3 response. The gap between them is the CLRT. The SEL-751 does this. *Combined ACK (Case B)*: other devices (an AB1400, an ION7550) piggy-back the acknowledgement onto the response — there is no standalone ACK, so no CLRT to measure. Case A is the whole game here because it is the case that *leaks* a CLRT fingerprint.

There are actually **two** passive leakage channels, and this project attacks both:

- **Timing** — the CLRT (and the surrounding inter-packet gaps). This is the Formby fingerprint.
- **Size** — how big responses are and how they are segmented. A device that always answers with a 134-byte frame looks different from one that answers with 37 bytes.

2. The idea: obfuscate in the network

The defence is to make the outstation's traffic *look the same regardless of which device it is* — normalize the size, and normalize/reshape the timing — **without changing the relay, the master, or the DNP3 bytes' meaning.** The natural place to do that is a device in the middle: a **programmable switch**.

What a programmable switch is (P4 and Tofino), plainly

An ordinary switch has fixed behaviour baked into silicon. A **programmable** switch lets you write the packet-processing logic yourself, in a language called **P4** [3], and compile it onto a high-speed chip. The chip we use is an **Intel Tofino-1**, whose architecture (a reconfigurable match-action pipeline, "RMT") came from Bosshart et al. [4]. The catch: this hardware processes packets at terabit rates by being *extremely* restricted — a fixed number of pipeline stages, no loops, no floating point, tiny per-packet state. Getting a "hold this packet for 13 milliseconds" behaviour out of a

chip designed to never hold anything is the central engineering tension of the project.

> **The inspiration: Ditto.** Meier, Lenders and Vanbever's *Ditto* (NDSS 2022) [5] showed that a programmable switch can obfuscate WAN traffic **at line rate** by padding packets to fixed sizes and releasing them on a deterministic schedule, so an observer sees a device-independent pattern instead of the real sizes and timings. Our project adapts that idea to DNP3 in a substation: pad the *size*, and reshape the *timing*, of a protection relay's responses. Ditto is the methodological ancestor of the queue/traffic-manager scheduling direction we pursue.

Within Case A there are two complementary timing defences:

Defence	Mechanism	Goal
Defence 1 — delay the ACK	Hold the pure TCP ACK; let the response go near its natural time	Shrink the ACK-to-response gap so the CLRT fingerprint collapses
Defence 2 — delay the response	Forward the ACK normally; release the response on a chosen schedule	Force the CLRT to a device-independent target, hiding the relay's native processing time

Both were previously demonstrated on the Tofino via a "recirculation-hold" mechanism (bouncing a packet through the pipeline to burn time) and are treated as a **frozen feasibility baseline**. The work in this report is the *next* layer: proving the size axis on real silicon, and — the bulk of what follows — replacing captured-file replay with the **physical relay** so that every timing number is measured against real hardware.

3. The testbed

Four machines and one relay, on a lab bench.

Vision runs the DNP3 master (it has a working `pydnp3` stack). **Hulk** drives high-rate traffic into the Tofino. **gambit** is the dev box where analysis runs. The **Tofino-1** is the programmable switch. The **physical SEL-751** is the real relay we finally connected. A hard rule governs everything: **the switch is shared and any change to it is gated on explicit human authorization**, and the relay is only ever *read*, never controlled or reconfigured by us.

4. Thread A — the size axis, proven on silicon

Before the relay work, we proved the **size** half of the obfuscation on the actual Tofino. The mechanism (Level-1): take a corpus of real DNP3 frame sizes and **pad every one of them to a single 128-byte state**, on-chip, in the dataplane.

On live Tofino-1 silicon, across three reproducible runs of 150 frames each: every output was exactly 128 bytes, with **zero loss and zero reordering**. The information the size channel gave away — measured as *mutual information*, in bits, between a frame's size and the device that sent it — went from **0.91 bits to 0.00 bits**. In plain terms: before, a frame's length told you something about which device sent it; after, it tells you nothing, because every frame is the same length.

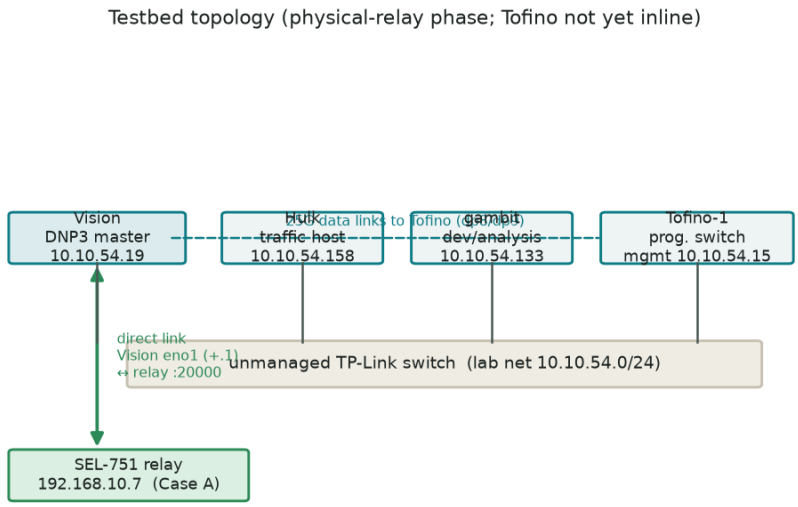

Figure 2: Testbed topology for the physical-relay phase. Vision (master) reaches the SEL-751 directly through the unmanaged lab switch; the Tofino is present but not yet placed inline.

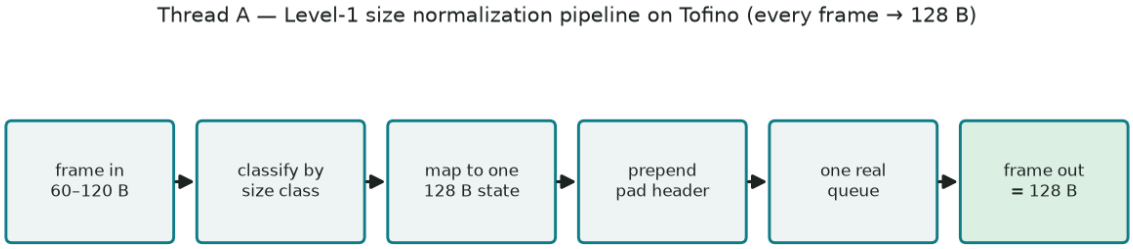

Figure 3: Thread A pipeline. Each frame is classified, mapped to one 128 B target state, padded with a compile-time pad header, placed on one real queue, and emitted — always 128 bytes.

> **Honest scope of Thread A.** This was a **Level-1** result: the switch classified frames by a *declared* size tag we attached, not by parsing live DNP3/TCP, and it addressed only the size channel on a small three-flow corpus. It is a genuine on-silicon proof that the padding mechanism works with no loss or reordering — not a claim that a full inline DNP3 defence is finished. Evidence: `research/tofino_dcrn_feasibility/p4/queue_microbench/autonomous_run_20260722/` (tag `queue-trace-level1-hw-pass`).

5. Thread B — bringing the physical relay online

Everything above used captured traffic replayed from files. The direction was to stop relying on replay and **connect the physical SEL-751**, first through the ordinary lab switch (with the Tofino *not* inline yet), just to establish a real, measured baseline. This turned into a multi-stage debugging story worth telling in full, because each stage taught something.

Challenge 1 — the relay was invisible. We plugged the relay into the lab switch and it simply did not appear. From Vision we ran an ARP scan of the whole `10.10.54.0/24` subnet and a 30-second passive capture: **zero** packets from any Schweitzer device. The reason is mundane but important: an **un-polled DNP3 relay is silent** — it does not announce itself, it only answers when spoken to. You cannot discover it passively; you must know its IP.

Challenge 2 — a wrong theory, corrected by evidence. Because the capture showed 802.1Q VLAN tags, I initially suspected the relay's switch port was on a different VLAN. Then the physical photos showed the switch is a **TP-Link TL-SG1024S — an unmanaged switch with no VLANs at all**. The tags were just other devices' traffic passing through. The VLAN theory was wrong; I retracted it. (This is a recurring theme: every theory got checked against evidence, and the wrong ones were dropped, not defended.)

Challenge 3 — the smoking gun: an accept-then-hang-up. The relay's real address turned out to be `192.168.10.7` on its own subnet. With Vision given a matching address, ping worked and TCP port 20000 opened — but every DNP3 session **died instantly**.

The relay accepted the TCP handshake and then closed the connection itself before any DNP3 was exchanged. The cause, once we read the relay's configuration, was a **DNP3 master-IP allowlist**: the relay setting `DNPIP1 := 192.168.10.1` means it only talks DNP3 to a master at `192.168.10.1`. Vision was at `.100`. Not on the list, so accepted then dropped.

> **Honest disclosure — I hammered the relay by accident.** The DNP3 library (`opendnp3`) defaults to auto-reconnecting when a channel drops. Because the relay closed every session instantly, the library reconnected **~55 times per second for ~8 seconds — 434 TCP sessions**. I intended one session; the retry loop produced hundreds. The mitigating fact, verified in the capture: **zero DNP3 application bytes were ever sent** across all of them — no read, no control, no write — so no protocol-level safety rule was violated. The lesson went straight into the fix: the next probe used a *no-retry* transport (a one-hour minimum reconnect interval), so a drop cannot trigger a reconnection. Documented in `research/physical_sel751/SEL751_DIRECT_CONNECTIVITY_REPORT.md`.

Challenge 4 — the DNP3 library's dangerous defaults. Talking to a live protection relay read-only is not just "don't send a control command." The `opendnp3` master, left on defaults,

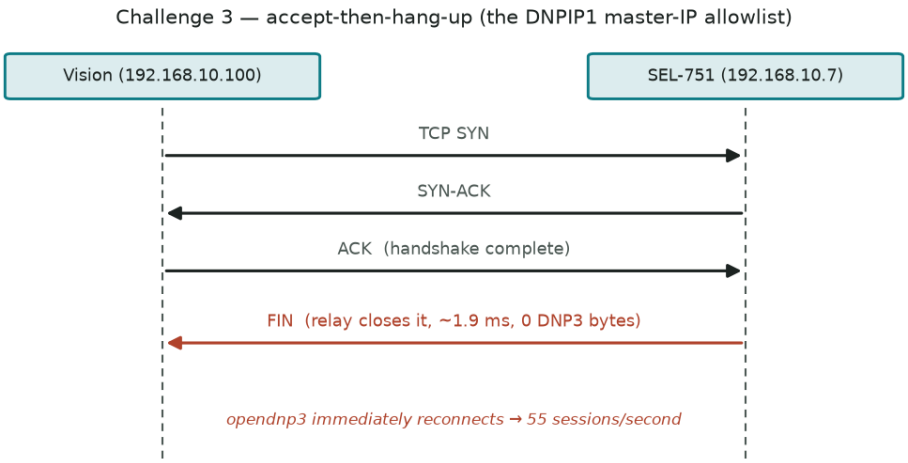

Figure 4: Challenge 3. The relay accepts the TCP handshake, then closes it itself ~1.9 ms later with no DNP3 exchanged; opendnp3's default auto-retry then reconnects dozens of times per second.

will *automatically*: send an `ENABLE_UNSOLICITED` request, send a `DISABLE_UNSOLICITED` request at startup, and — most dangerously — **send a WRITE to clear the relay's "device restart" flag**. A WRITE to a protection relay is exactly what a read-only experiment must never do. So the probe pins every automatic behaviour off: no startup poll, no unsolicited management, no time-sync, and `ignoreRestartIIN = True` so the restart flag is never cleared. The verified-safe probe is `research/physical_sel751/native_class0_probe.py`.

The payoff — a clean native transaction. With Vision at `.1`, outstation address `0` (its real configured value — not the `10` from the old captures), and the no-retry probe, the relay answered on the first try. One TCP session, one Class-0 read, a separate pure ACK, then a 134-byte response carrying 69 points. **Case A confirmed on the physical device**, with a first-transaction CLRT of 6.12 ms.

6. The 300-poll CLRT experiment

One transaction is an anecdote, not a distribution. The authorized experiment: **300 sequential Class-0 reads over one persistent TCP session**, one request outstanding at a time, a one-second pause after each response, **no retries, no reconnects**, read-only, with hard stop conditions (any reset, timeout, unexpected function, or protocol error ends it immediately). It ran ~5 minutes and completed all 300 with no stop condition, one TCP session, zero resets, zero retransmissions. The probe is `clrt_300poll_.../clrt_experiment.py`; the raw evidence and a SHA-256 manifest are in that directory.

> **Worked example — what "CLRT" is, in numbers.** For each poll we timestamp three wire events: the request leaving Vision, the relay's bare TCP ACK, and the relay's DNP3 response. Then `request->ACK = 0.9 ms`, `CLRT = ACK->response = 6.1 ms`, `request->response = 7.0 ms` for that first poll. The CLRT — the middle number — is the Formby fingerprint. We compute it 300 times and study the *distribution*.

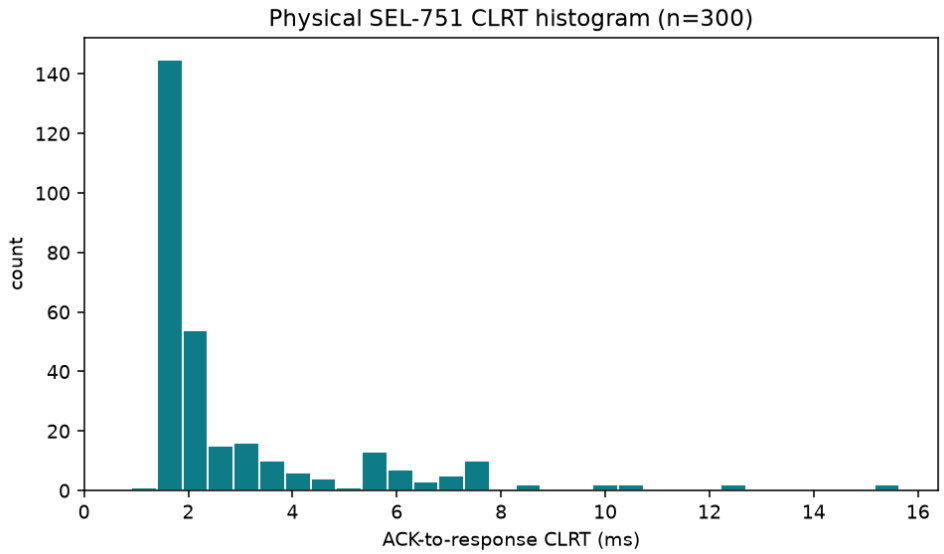

Figure 5: Figure 1. Distribution of the CLRT over 300 polls. Most responses cluster tightly around ~1.9 ms, with a right-hand tail out to ~15.6 ms. The long thin tail is why the mean (2.98 ms) sits above the median (1.90 ms): a handful of slow responses drag the average up.

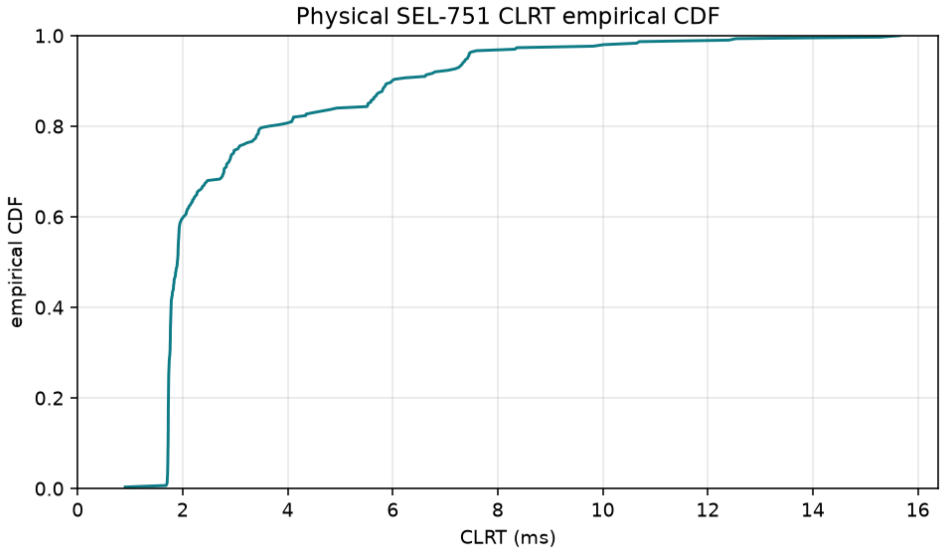

Figure 6: Figure 2. Empirical CDF of the CLRT — "what fraction of polls had a CLRT at or below x?" The curve rises almost vertically near 1.9 ms (most polls are there), then crawls rightward through the slow tail. Half the mass is below 1.90 ms (median); 90 percent below ~6.0 ms; 95 percent below ~7.4 ms.

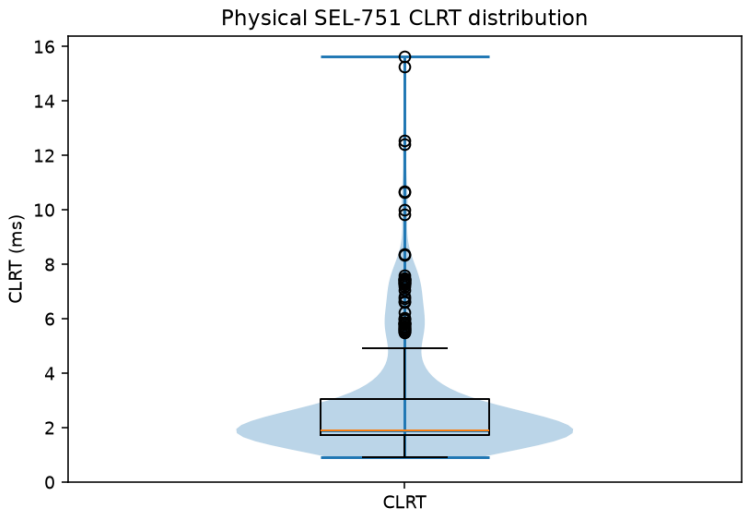

Figure 7: Figure 3. Box-and-violin view. The violin's width shows where the data pile up (a fat lobe at ~1.9 ms); the box marks the 25th-75th percentiles (1.73-3.06 ms) with the median inside; the points above are the slow-tail outliers.

CLRT statistic (n=300)	value (ms)	plain meaning
median	1.899	the typical response time
mean	2.983	the average, pulled up by the tail
std dev	2.273	spread around the mean
p90 / p95	5.99 / 7.43	9-in-10 / 19-in-20 are faster than this
min / max	0.905 / 15.649	fastest / slowest single poll

The bug that hid inside the framing

The first analysis pass mis-counted the transactions by one. The cause is a nice illustration of DNP3 framing: **every** DNP3 frame — including pure link-layer housekeeping frames that carry no application data — begins with the two magic bytes `0x05 0x64`. When a fresh TCP session opens, the relay and master exchange a link-status handshake (two such frames) before any real read. My parser counted the master's link-status frame as if it were a request, shifting every poll's data by one. The fix was to require an actual application layer (a link length field greater than 5) and the correct application function code before treating a frame as a request or response. After the fix: exactly 300 requests, 300 responses, none missing.

7. The validation pass

A distribution is only as trustworthy as the assumptions behind its summary statistics. The validation pass — run entirely on the already-committed evidence, changing no raw data — stress-tested four things.

7.1 Decoding what the relay actually said (the IIN field)

Every DNP3 response carries two "Internal Indication" bytes — status flags from the outstation. Ours read `0x80 0x00` on all 300 responses. The report had rendered this as "0x8000," which is *endian-ambiguous*: is the set bit in the first byte or the second?

> **Worked example — reading the IIN bits.** On the wire the first byte is **IIN1 = `0x80`** = binary `1000 0000` = only bit 7 set. In DNP3, IIN1 bit 7 is **DEVICE_RESTART**. The second byte is **IIN2 = `0x00`** = no bits, and critically none of IIN2's bits are *error* bits. So the relay is saying, on every response: "I restarted at some point, and I have no error." The restart flag stays lit because a normal master would clear it with a WRITE — and we deliberately never write. The corrected, unambiguous notation used everywhere now is `IIN1=0x80, IIN2=0x00`. Reproducible via `validation/validate_iin.py`.

7.2 Are the samples independent? (They are not.)

Every statistic that follows — confidence intervals especially — silently assumes the 300 CLRT values are *independent* draws. We tested that with an **autocorrelation** analysis: does a slow poll tend to be followed by another slow poll?

7.3 Fixing the confidence intervals (bootstrap)

> **What a bootstrap is, in one paragraph.** A *bootstrap* estimates how uncertain a statistic is by resampling the data itself: draw 300 values (with replacement) from your 300, recompute the median, repeat 10,000 times, and look at the spread of those medians. The middle 95 percent of that spread is a 95 percent confidence interval — *without* assuming the data follow any particular distribution. Its catch: the ordinary bootstrap assumes the observations are **independent**. We just showed ours are not.

Because the CLRT is autocorrelated, the ordinary ("IID") bootstrap treats bursty, correlated data as if it carried more independent information than it really does, so its intervals come out **too narrow**. The fix is a **moving-block bootstrap**: resample *contiguous blocks* of consecutive polls instead of individual polls, so each block keeps its internal correlation. The point estimates do not change; the honest intervals widen:

CLRT statistic	IID bootstrap 95% CI	moving-block (L=7)	moving-block (L=30)
mean	[2.73, 3.25]	[2.59, 3.40]	[2.36, 3.65]
median	[1.82, 1.93]	[1.79, 2.06]	[1.78, 2.29]

The median interval roughly **doubles-to-triples** under the correct method. The lesson, stated plainly in the reports: *the originally-quoted CIs were anti-conservative; the moving-block intervals supersede them for any uncertainty statement.* Full write-up: `validation/TEMPORAL_DEPENDENCE_ANALYSIS.md`.

7.4 The historical "~13 ms" mystery

Prior project documents and the direction cite a native SEL-751 CLRT of ~**13 ms**, from earlier captured traces. Our live median is **1.9 ms** — a 7x difference for the same device and the same measurement. That gap had to be explained, not brushed aside.

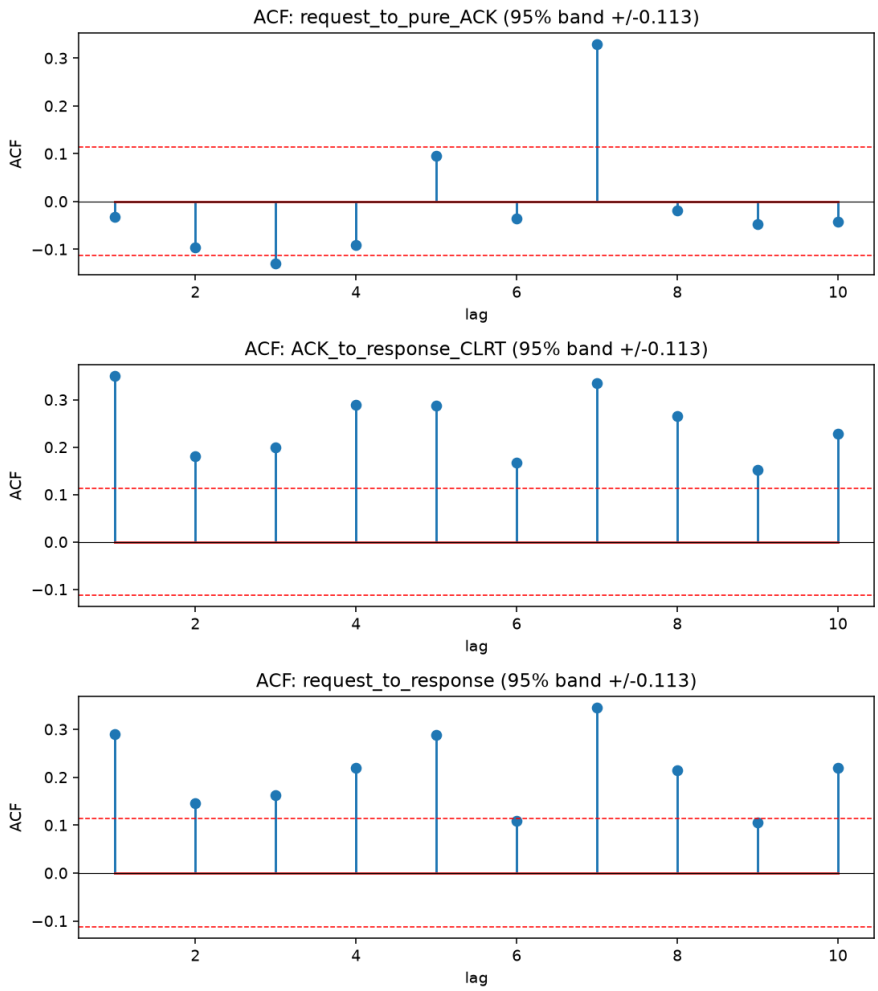

Figure 8: Figure 4. Autocorrelation (ACF) at lags 1-10 for all three timing series. Each bar asks: how correlated is a value with the value k polls earlier? The red dashed lines are the 95 percent "no real correlation" band. For the CLRT (middle panel) the lag-1 bar is 0.35 and every bar 1-10 pokes above the band — strong, persistent positive correlation. Slow responses come in bursts, not at random. The Ljung-Box test returns p ~ 0, overwhelmingly rejecting independence.

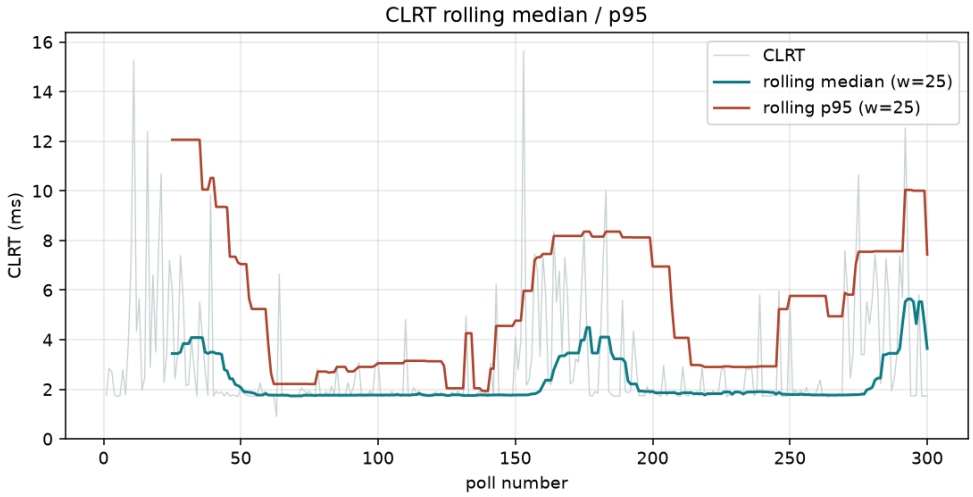

Figure 9: Figure 5. Rolling median and rolling p95 (window = 25 polls). The median (teal) is flat across the run — the typical response time does not drift. But the p95 (red) rises and falls, highest in the first ~50 polls: the tail is bursty and slightly heavier early on.

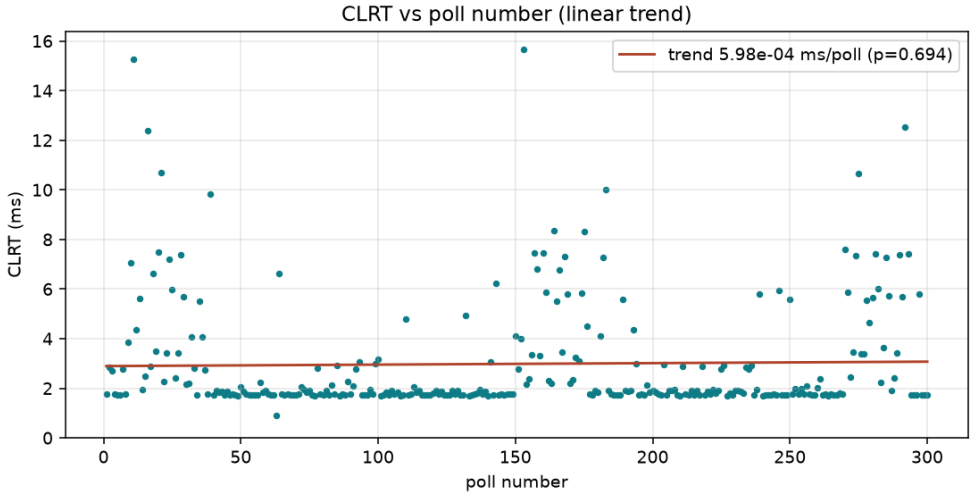

Figure 10: Figure 6. CLRT versus poll number, with a fitted straight line. The line is essentially flat (slope ~ 6e-4 ms per poll, p = 0.69, r-squared ~ 0.0005): there is no drift or warm-up trend over the five minutes. The dependence is short-range clustering, not a slow ramp.

Recomputing directly from the original trace (`Traffic Trace/SEL751.pcap`) reproduced the old number *exactly*: median 12.90 ms over 299 transactions, and it genuinely is the ACK-to-response CLRT. Then the decisive test — split it by request type:

dataset	request	n	resp bytes	CLRT median (ms)	req->ACK median (ms)
historical	DIRECT_OPERATE	200	37	12.84	3.67
historical	READ	99	54	13.18	3.83
live 300-poll	READ (Class-0)	300	134	**1.90**	**0.56**

This kills the obvious hypothesis. The historical *READ-only* CLRT (13.18 ms) is essentially the same as its control CLRT (12.84 ms), so the gap is **not** caused by the historical traffic being control-heavy. Instead, the whole historical environment was uniformly ~7x slower — in *both* the req->ACK (3.7 vs 0.56 ms) *and* the CLRT — which points to a systematic difference (network path, capture point, relay firmware/config/load, or a different setup entirely), not a per-request effect.

> **A tempting inference I checked and retracted.** I briefly suspected the historical `10.0.0.1` was a *simulator*, because its packets carried IP TTL 64 (a Linux signature) rather than 255. Then I checked the live physical relay's own TCP packets in the committed capture — they are **also TTL 64** (its ping replies are 255, but its TCP is 64). So TTL does *not* distinguish the two, and the simulator inference was wrong. I struck it. What honestly remains: the ~13 ms is a real CLRT from a different, undocumented capture context; the 1.9 ms is the physical relay's CLRT in the current direct setup; and **the cause of the offset is undetermined from the available evidence.** The two numbers should not be compared head-to-head. Full analysis: `validation/HISTORICAL_13MS_RECONCILIATION.md`.

8. Challenges and lessons

Challenge	What it actually was	Lesson / fix
Relay invisible on the wire	An un-polled DNP3 relay is silent; no gratuitous ARP	Discovery needs the IP; passive scanning cannot find it
"Different VLAN" theory	Switch is unmanaged (no VLANs); tags were other traffic	Check theories against the hardware; retract when wrong
Accept-then-hang-up	Relay master-IP allowlist (`DNPIP1`) excluded us	Read the device config; match the master IP
Accidental 434-session storm	opendnp3 auto-retry vs. a relay that closes instantly	Use a no-retry transport for controlled single sessions
Library's unsafe defaults	Auto WRITE to clear restart-IIN; auto unsolicited mgmt	Pin every automatic behaviour off for a read-only probe
Off-by-one in analysis	Link-layer frames also start `0x0564`	Require an app layer + correct function code before counting

Challenge	What it actually was	Lesson / fix
Over-narrow confidence intervals	CLRT is autocorrelated; IID bootstrap invalid	Moving-block bootstrap; report both, prefer the block CI
~13 ms vs 1.9 ms	Not request-type; a ~7x systematic environment offset	Reproduce, split, and state honestly what is undetermined

9. Where this sits in the literature

The project lives at the intersection of three research lines. **ICS device fingerprinting** is the threat: Formby et al. [2] established the CLRT as a hard-to-hide physical fingerprint of control-system devices — the exact signal our timing defences target, and the exact number this report measures on a real relay. **Traffic-analysis defences** are the method: the website-fingerprinting community learned, painfully, that naive padding and simple timing tricks are defeated by better classifiers — Dyer et al.'s "Peek-a-Boo" [6] showed coarse countermeasures fail; principled defences like Wright et al.'s traffic morphing [7], Cai et al.'s Tamaraw [8], and Juarez et al.'s WTF-PAD [9] shape both size and timing with explicit cost/robustness trade-offs. That literature is *why* we insist on measuring information leakage (mutual information, classifier accuracy) rather than eyeballing "it looks obfuscated." **Programmable dataplanes** are the platform: P4 [3] and the RMT architecture [4] behind Tofino make line-rate, in-network shaping possible, and Ditto [5] is the direct precedent for doing size+timing obfuscation on such a switch. Our specific contribution is to bring this to a **non-cooperative, real DNP3 protection relay**, and to hold ourselves to measured, statistically-honest results at every step.

10. Limitations and what's next

- **Size axis:** proven on silicon but at *Level-1* (declared size class, not live DNP3 parsing), single small corpus.
- **Timing axis:** the two Case-A defences exist as a frozen recirculation-based feasibility baseline; the queue/traffic-manager version inspired by Ditto is designed, not yet built.
- **Physical relay:** reachable, Case A confirmed, CLRT distribution measured and validated. But: one relay, one configuration, one 1 Hz session, 300 samples, read-only Class-0 only; no strong tail (p99) claim; CLRT is load-sensitive and load was uncontrolled.
- **Inline defence:** the Tofino has *not* yet been placed between the master and the physical relay — that is gated on explicit authorization and is the natural next step (a "shadow mode" that parses live DNP3 and measures without modifying, before any active padding/holding).
- **The ~13 ms offset** between the historical traces and the live relay is *undetermined*; resolving it needs the original capture's provenance and a controlled A/B on the same physical relay.

Appendix A — file and evidence map

- **Physical relay (Thread B)** — `research/physical_sel751/`
  - `SEL751_DIRECT_CONNECTIVITY_REPORT.md` — the connectivity saga + native baseline
  - `native_class0_probe.py` — the verified-safe single-poll probe
  - `evidence/native_class0_v2.pcap` — the first clean transaction

  - `clrt_300poll_20260723T152242/` — the 300-poll experiment (`clrt_experiment.py`, `analyze_clrt.py`, `per_poll.csv`, `summary.{csv,json}`, `CLRT_EXPERIMENT_REPORT.md`, `plots/`, `evidence/`, `SHA256SUMS.txt`)
  - `.../validation/` — IIN, temporal-dependence, and historical-reconciliation reports + scripts + plots
- **Size axis (Thread A)** — `research/tofino_dcrn_feasibility/p4/queue_microbench/autonomous_run_` (`HARDWARE_RESULT.md`, evidence; tag `queue-trace-level1-hw-pass`)
- **Timing defences (frozen baseline)** — `research/tofino_dcrn_feasibility/p4/ack_delay/` (`dcrn_defense1/2.p4`, `ACK_DELAY_*.md`, `evidence/clrt_baseline.py`)
- **Original device traces** — `Traffic Trace/SEL751.pcap` (and AB1400, ION7550)
- **Direction / meeting** — `meeting.md`, `meeting_direction.md`

Appendix B — references

1. IEEE Standards Association. *IEEE Std 1815-2012 — IEEE Standard for Electric Power Systems Communications — Distributed Network Protocol (DNP3).* IEEE, 2012. (The DNP3 protocol standard.)
2. D. Formby, P. Srinivasan, A. M. Leonard, J. D. Rogers, and R. Beyah. "Who's in Control of Your Control System? Device Fingerprinting for Cyber-Physical Systems." *Network and Distributed System Security Symposium (NDSS)*, 2016. (Introduces Cross-Layer Response Time fingerprinting. Verified via Semantic Scholar.)
3. P. Bosshart, D. Daly, G. Gibb, M. Izzard, N. McKeown, J. Rexford, C. Schlesinger, D. Talayco, A. Vahdat, G. Varghese, and D. Walker. "P4: Programming Protocol-Independent Packet Processors." *ACM SIGCOMM Computer Communication Review*, 44(3), 2014. (The P4 language. Verified; Semantic Scholar lists the 2013 preprint / CCR record.)
4. P. Bosshart, G. Gibb, H.-S. Kim, G. Varghese, N. McKeown, M. Izzard, F. Mujica, and M. Horowitz. "Forwarding Metamorphosis: Fast Programmable Match-Action Processing in Hardware for SDN." *ACM SIGCOMM*, 2013. (The RMT architecture behind Tofino. Verified.)
5. R. Meier, V. Lenders, and L. Vanbever. "Ditto: WAN Traffic Obfuscation at Line Rate." *NDSS*,
   2022. (In-network size+timing obfuscation on a programmable switch. Verified.)
6. K. P. Dyer, S. E. Coull, T. Ristenpart, and T. Shrimpton. "Peek-a-Boo, I Still See You: Why Efficient Traffic Analysis Countermeasures Fail." *IEEE Symposium on Security and Privacy*,
   2012. (Why naive padding/timing defences are defeated. Verified.)
7. C. V. Wright, S. E. Coull, and F. Monrose. "Traffic Morphing: An Efficient Defense Against Statistical Traffic Analysis." *NDSS*, 2009. (Shaping one traffic distribution to look like another. Well-established; not re-verified this session — Semantic Scholar was rate-limited.)
8. X. Cai, R. Nithyanand, T. Wang, R. Johnson, and I. Goldberg. "A Systematic Approach to Developing and Evaluating Website Fingerprinting Defenses" (Tamaraw). *ACM CCS*, 2014. (Provably-bounded size+timing padding. Verified.)
9. M. Juarez, M. Imani, M. Perry, C. Diaz, and M. Wright. "Toward an Efficient Website Fingerprinting Defense" (WTF-PAD). *ESORICS*, 2016. (Adaptive padding to break timing features. Verified; Semantic Scholar lists the 2015 preprint.)

Prepared with AI assistance (Claude Code). Every measured number in this report traces to a committed evidence file with a SHA-256 manifest; every cited paper was checked for existence against Semantic Scholar except where noted.